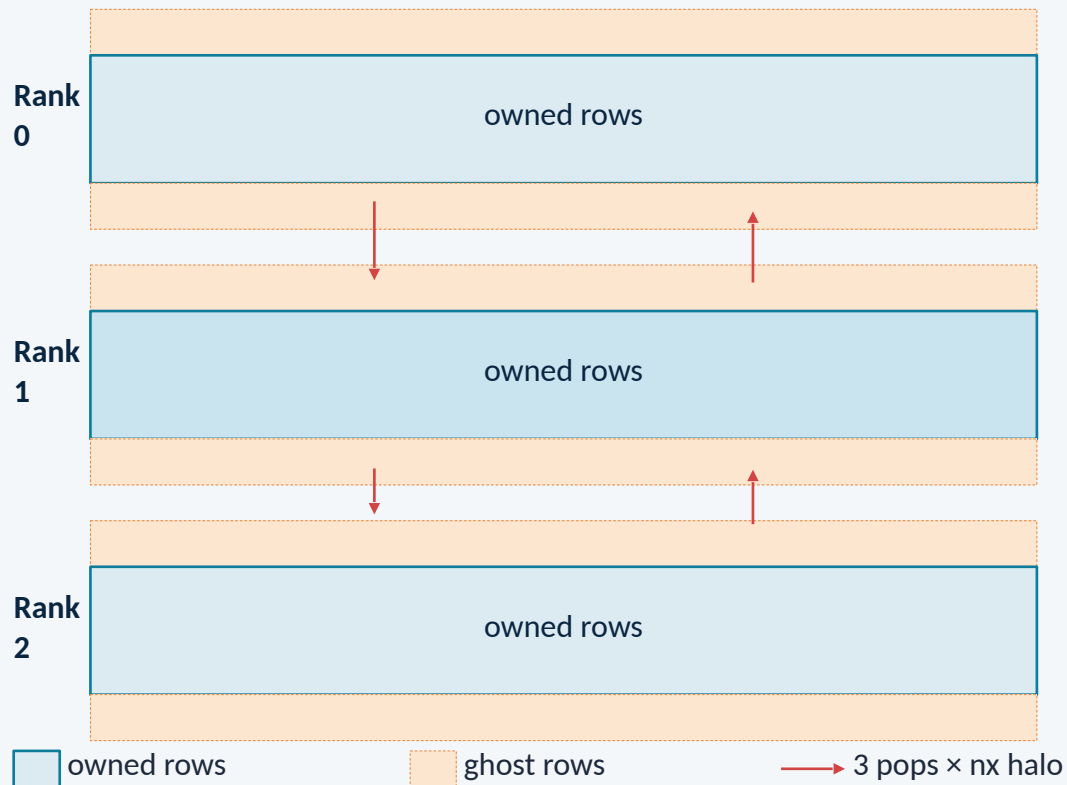


MPI Design — 1D Row Decomposition

Halo exchange · communication/computation overlap · pull streaming

Domain decomposition (D2Q9, n_y split across ranks)



Decomposition

1D row-block split of the n_y rows; each rank owns a contiguous slab plus one ghost row above and below.

Halo exchange

Non-blocking MPI_Irecv / MPI_Isend with upper and lower neighbours. Only the 3 populations crossing each boundary are sent — $3 \cdot n_x$ doubles instead of $9 \cdot n_x$.

Compute / comm overlap

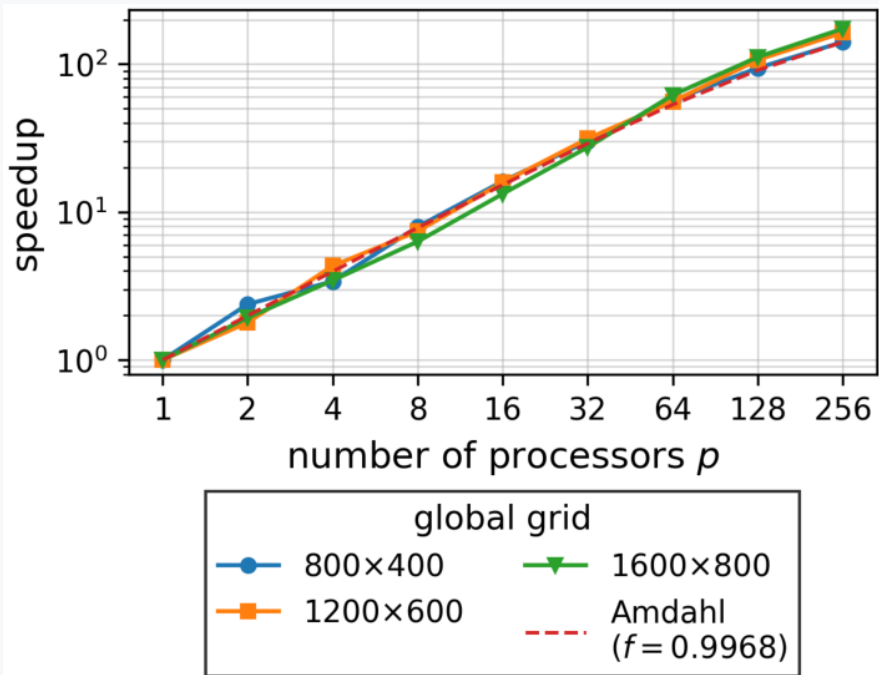
Collide + bounce-back on owned rows → post halos → stream interior rows while messages fly → MPI_Waitall → stream boundary rows with received ghosts.

Correctness

validation recovered $St = 0.1600$ for $p = 1 \dots 256$.

MPI Scaling: strong near Amdahl, weak limited by halos

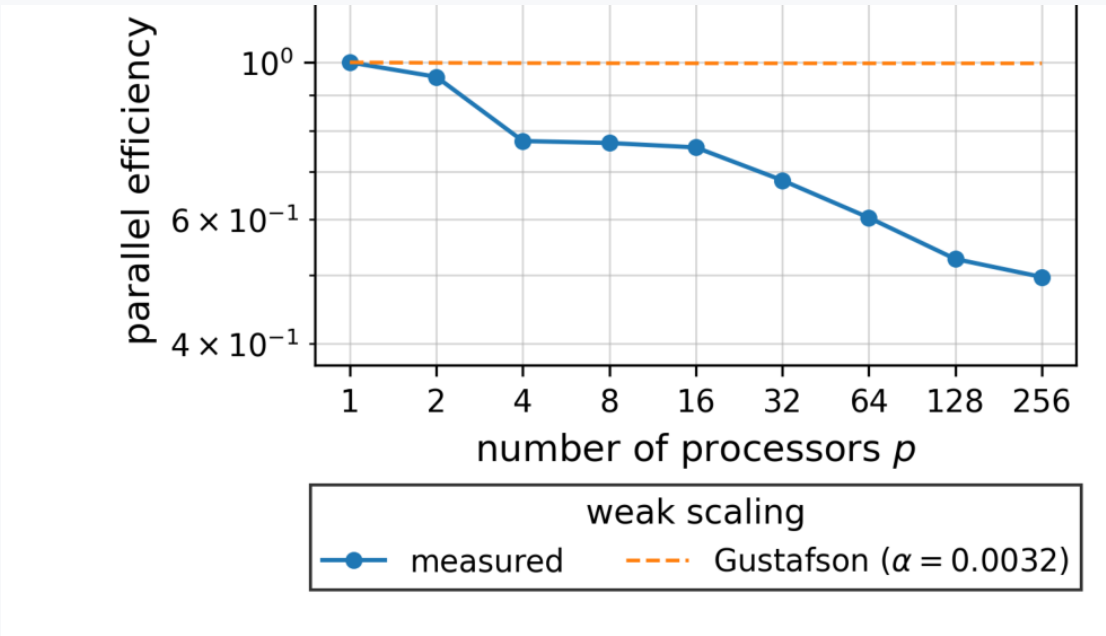
Strong scaling



p = 256 speedup
800x400: 141x
1200x600: 163x
1600x800: 174x

Amdahl reference
 $S(256) \approx 141x$

Weak scaling



weak efficiency
95.4% $\rightarrow$ 49.7%

main bottleneck
halo + sync

Observation: larger grids exceed the simple Amdahl curve because smaller per-rank blocks improve cache locality.
Limit: at high p , 1D row decomposition raises communication relative to local work; $p = 256$ communication reaches 70.8% on 800x400.
Weak scaling: Gustafson predicts near-ideal efficiency, but measured efficiency falls as halo exchange and synchronization dominate.

CUDA — Fused Pull-Streaming + BGK Collision

Fused kernel

One thread per destination cell pulls the 9 incoming D2Q9 populations into registers, computes p and u , applies BGK, and writes f at $t+1$.

4907

Best CUDA (MLUPS)

1500×750, 64×4, pa=64

187.7×

vs serial baseline

800×400: 21.20 → 3979.15
MLUPS

78 %

of bandwidth roofline

6 250 MLUPS @ 900 GB/s

Memory layout — SoA

$f[q \cdot (\text{pitch} \cdot n_y) + y \cdot \text{pitch} + x]$. Threads adjacent in x access contiguous memory for the same population $q \Rightarrow$ coalesced loads. Row pitch padded to `pitch_align`.

Correctness

$St = 0.1600$ across all nine block $\times$ pitch-alignment configurations



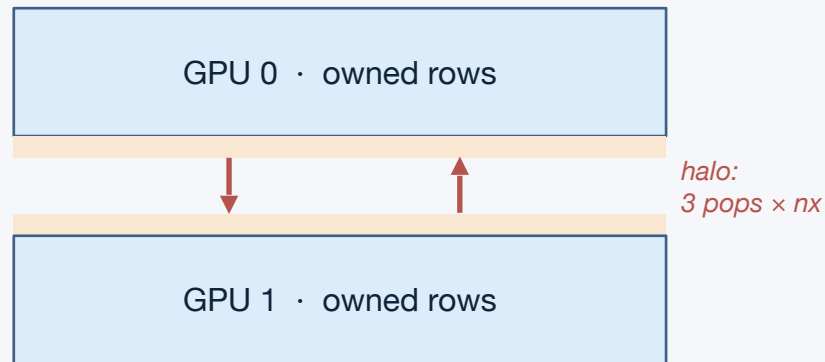
No intermediate f^* array in global memory; one thread owns one output cell.

Takeaway: memory-bandwidth limited; wider x -blocks improve coalescing, larger grids amortise launch overhead.

Going Further — Single-Node Hybrid MPI-CUDA

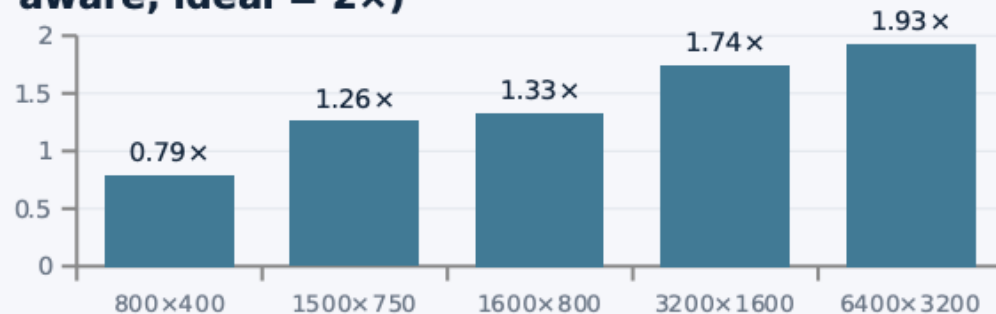
Two GPUs on one node · fused pull-stream/BGK kernel reused · only the halo exchange changes

Domain split across 2 GPUs · 1 rank $\leftrightarrow$ 1 GPU



peach shading = exchanged ghost rows

2-GPU vs 1-GPU speedup, same grid (CUDA-aware, ideal = 2x)



Why a hybrid

MPI-only does the update on CPUs:

- few ranks are compute-bound and slow
- many ranks pay heavy halo + sync cost.

CUDA-only is fast but capped by a single device's memory and bandwidth.

Design

A few MPI ranks, each bound to one GPU, own a row-block + 2 ghost rows and reuse the fused pull-stream/BGK kernel unchanged. Only the halo changes: exchange the 3 D2Q9 pops crossing each boundary. Tested CUDA-aware (device buffers) and host-staged (pinned) — no NVLink / GPUDirect assumed.

Results

Benefit grows with grid. At 6400x3200: 10138 MLUPS, 1.93x speedup, 96.4% efficiency — beating best single-GPU CUDA (4900) and MPI-only (3998). Small grids regress (800x400: 0.79x) as halo dominates; host-staged tracks CUDA-aware, so halo overhead is the limit, not staging.